Distilling a Language-Model Decision Policy into Browser-Scale Students

How a small model of structured triples can stand in for a three-billion-parameter decision model in a town-building game

Shishir Tandale, with Claude

ABSTRACT

Hearthvale is a town-building game in which every resident's choices, from the daily routine to emigration and political protest, are made by a hosted decision model, Liquid AI's d1-3B. The model gives towns varied and believable behaviour, but its cost grows with every player and with game speed, and a single shared GPU soon becomes the limiting resource. Because the game's questions are highly structured, a small model in each player's browser can take its place. Building on the Triple World Model, an unpublished research system that represents state as sets of entity, attribute and value triples, we write each question as an unordered set of triples and distil the teacher into a compact set model: every fact is one token, a town's facts are projected once into the keys and values its questions attend to, and every permitted answer is a token that ends in a score. With about 118,000 parameters the student reproduces the teacher's answer distributions to within 0.021 nats on towns it never saw, generalizes to combinations of facts withheld from training far better than a conventional multilayer perceptron, and answers a question in about 70 microseconds on one processor core. Towns under its control, played by a simulated player, follow the teacher's towns from the middle of the game on and vary as much; in their first decades they grow more slowly, a lag we trace to a single decision in small towns. A learned estimate of the student's own error drives a daily training loop that learns from players' questions within per-player limits and publishes a new version only when it passes a set of comparisons with the current one.

1 Introduction

1.1 A town that decides for itself

Simulation games have long relied on hand-written rules to animate their populations. Hearthvale takes a different course. The player governs a town as its council, reading a docket of proposals and ruling on them, while the town itself makes thousands of smaller decisions: whether a resident works or rests this morning, whether a household stays, moves or sends for relatives, whether a faction backs a new tax. Each of these is put to a decision model, Liquid AI's d1-3B, which reads a short description of the town and a question about a group of residents and returns a probability for every permitted answer. The game samples from that distribution, so a district of labourers might divide between work and rest in proportions that shift with the season, the food supply and the council's latest ruling. The result is behaviour that feels considered rather than scripted, and it is central to the game's character.

1.2 The cost of a hosted policy

This realism has a price that grows with success. Every question occupies time on a shared GPU, and the volume of questions rises with the size of each town, with the speed at which players run the game, and with the number of players. A city asks about eighteen questions a second at normal speed and several hundred at the highest speeds. Three costs follow. GPU time limits how many players one server can support. Network delay means answers arrive late, after the game has already moved on. And when the server falls behind, the game reverts to simple fallback rules, which are not a neutral substitute: towns governed by them grow several times larger than towns governed by the model, so a busy server visibly changes how the game plays.

Removing the decision model would remove these costs, and with them the quality that justifies it. The aim is instead to keep its behaviour while moving most of its work elsewhere. A student model running in each player's browser answers most questions without the server, without delay and without a network connection at all. Because the student's purpose is to stand in for the teacher, it must copy the teacher's behaviour rather than invent its own, and so it learns only from the teacher's answers, never from outcomes in play.

1.3 Aim and contributions

The questions themselves suggest that this is feasible. Almost every input is categorical, such as whether food is low, whether rent is high or which institutions are active, and the same twenty-two kinds of question recur millions of times. A function with such a regular domain should be learnable by a model small enough to run in a browser. This report asks whether such a student can reproduce the teacher closely enough that a town under its control behaves like a town under the teacher, at a cost an ordinary device can bear.

We make four contributions. First, we adapt the Triple World Model's representation of state as sets of triples to decision questions, and argue from its properties why this representation suits the problem. Second, we compare five families of student model, and the parts of the best of them, on accuracy, cost and compositional generalization. Third, we propose an evaluation that goes beyond average error: it measures the student on the states its own policy reaches and compares complete towns with the teacher's in both their average course and their spread. Fourth, we describe how the student keeps learning from the teacher after release without ever changing beneath a game in progress.

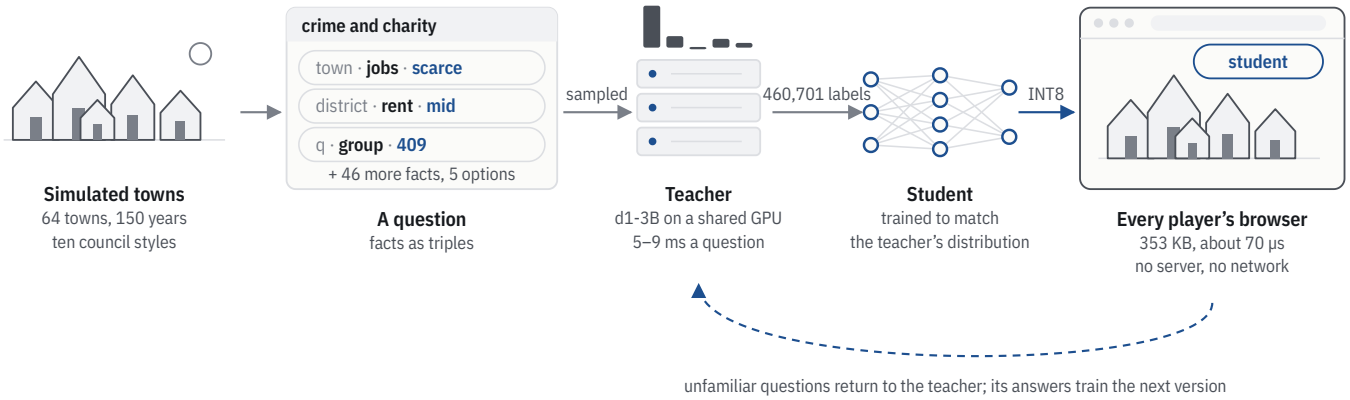


Figure 1. Overview of the approach. Questions gathered from simulated towns are labelled by the teacher, the student is trained on the labels and runs in every player's browser, and questions the student finds unfamiliar return to the teacher for the next version.

2 Background

2.1 Distillation

The idea of training a small model to imitate a large one is well established. Hinton and colleagues showed that a compact network trained on the full output distribution of a larger network, rather than on hard labels alone, retains much of its accuracy, because the distribution conveys how the teacher weighs the alternatives [1]. Rusu and colleagues carried the same idea to agents that act, distilling reinforcement-learning policies into smaller networks [2]. Our setting is a direct descendant of policy distillation, with one difference that shapes everything else: the teacher is a language model, and its inputs are structured records rather than images or game screens.

2.2 Learning on structured records

Structured records place the problem in the territory of tabular learning, where gradient-boosted decision trees such as XGBoost remain difficult to beat [3]. Neural approaches to tables commonly treat each column as a

token and let a transformer relate them, as in FT-Transformer [4]. A known weakness of plain multilayer perceptrons is that they learn multiplicative interactions between features only with difficulty [5], which is why recommendation systems add explicit interaction terms. Factorization machines express every pairwise interaction as a product of learned feature vectors, and can therefore estimate the effect of a pair never observed together [6]. Set models, finally, treat their input as an unordered collection and are invariant to its order by construction [7].

2.3 Distribution shift and deferral

Two further strands bear on deployment. Imitation learning has long recognized that a copied policy visits states its teacher never showed it, and that its errors compound there; the DAgger algorithm addresses this by collecting states from the student's own behaviour and labelling them with the teacher [8]. Selective prediction, in turn, allows a model to defer uncertain cases to a stronger one [9]. Both ideas reappear in the system, the first in how training data is gathered and the second in how the student consults its teacher.

2.4 The Triple World Model

The work reported here grows out of an earlier research system, the Triple World Model, which has not been published and is therefore described in some detail. Its premise is that a small model can reason well about a world if the world is presented to it as structure rather than as prose. A state is written as an unordered set of triples, each naming an entity, one of its attributes and the attribute's value, such as a pet, its hunger and the level of that hunger. A compact transformer reads all the triples at once and predicts the next state, producing every output triple simultaneously rather than one after another, and learning the change from the current state rather than the state afresh.

The system was built to test one claim: that this decomposition allows generalization to combinations of facts never seen together in training. On a benchmark spanning three domains, it predicted the outcome for novel combinations of entity and state with an F1 score of 0.74, and ablations showed that attention across positions was necessary to do so. Where the correct output depended on other triples present in the state, it reached an F1 of 0.978, and a model of 178,000 parameters matched one of 4.5 million. The same architecture, reduced to 29,000 parameters, ran a pet simulation entirely in the browser in 303 kilobytes of JavaScript with 98.9% exact agreement. The underlying argument is that structural priors, namely decomposition into triples, invariance to order and a shared embedding space, carry knowledge that an unstructured model would otherwise have to acquire through parameters and data.

2.5 Decision models

A recent class of models answers typed questions about a document with a probability distribution over the permitted options, in one forward pass and without generating text, behind a common interface its vendors call System One. Liquid AI's d1, the teacher in this work, has 3.12 billion parameters. TypeSafe's Jev is closed. Cloudflare's Clef family includes Clef-flash, a 9-billion-parameter model derived from Qwen3.5-9B [10]. Kev, from Jared Palmer, comes in sizes from 0.8 to 27 billion parameters and adds a low-rank adapter and a pointer head that scores each option against the question to a Qwen base model [11]. Laya, from Convai Innovations, has 421 million parameters and is built on ModernBERT-large [12]. These models are general: they read free text and answer any question put to them. The student described here makes the opposite trade. At about 118,000 parameters it is three to five orders of magnitude smaller than any of them and answers only Hearthvale's twenty-two kinds of question, from structured input, but it needs no server. The difference matters at the scale

of a game: at the hosted prices of October 2026 a million questions cost about $6 from d1 and $11 to $13 from Clef-flash, Jev or d1-ultra, and a single city at normal speed asks some 65,000 questions an hour.

3 Problem Setting

Each decision question consists of a kind, a shared snapshot and a short context. The kind names the decision, for instance a resident's routine, a household's choice to stay or leave, or a faction's stance on a proposal. The snapshot summarises the town, and for some kinds a district or a proposal, in coarse bands that change from one season to the next; many questions share the same snapshot. The context identifies who is asking: their trade, wealth, stage of life and immediate needs. The teacher's answer is a probability distribution over a fixed set of options for that kind, of which a question may permit a subset. Because the game samples from this distribution, a student must reproduce all of it rather than merely its most likely option, and we therefore measure agreement by the Kullback–Leibler divergence from teacher to student, expressed in nats.

A satisfactory student must meet three conditions. It must answer close to the teacher, including in towns it has never seen, since players build towns no training run anticipated. Towns under its control must behave like towns under the teacher, which is a stronger requirement than low average error, because the game samples from the student at every step for a century of game time and small biases accumulate. And it must be small and fast enough to run within the game's existing budget on an ordinary device, including the bursts of questions that open each game month.

4 Approach

4.1 Why the triple route

Hearthvale's decision questions are already structured records: a handful of categorical facts about a town, a district and the residents in question. The Triple World Model was designed for exactly this kind of input, and its properties correspond closely to the requirements set out above. Table 1 summarises the correspondence; the paragraphs below explain the most consequential points.

Table 1. Properties of the triple representation and the requirement each one serves.

Property	What it means here	Requirement it serves
Shared vocabulary	Every kind of question is written in the same terms, so one model answers all twenty-two	Size; maintenance
Order-free sets	Facts have no natural order, and lists of rules and institutions vary in length	Accuracy on varied towns
Multiplicative interactions	Attention relates pairs of facts, such as a faction and the interests a proposal serves	Accuracy
Compositional generalization	Answers remain sound for combinations of circumstances absent from training	Unseen towns
Small parameter count	The architecture has already run in a browser at tens of thousands of parameters	Device budget
Extension by tokens	A new fact is a new token and a new kind needs no feature code; the vocabulary can grow without retraining from scratch	Learning after release
A cacheable state	The snapshot, shared by many questions, can be encoded once and reused	Device budget; bursts

Two of these properties weigh most heavily. The first is compositional generalization. Players control the most consequential part of the input, the rules in force, and combine them in ways no simulated council ever did; a student that merely memorises combinations seen in training would fail precisely where the game depends on it. The Triple World Model's central result was generalization of this kind, and it depended on attention between facts rather than on a fixed function of their sum. The second is economy. A single representation for every kind of question, derived from the schemas

the game already uses, means one small model rather than twenty-two, one runtime in the browser and no feature engineering when a kind is added or changed.

The decision problem also differs from the world-modelling problem in ways that require adaptation. A world model predicts a whole next state, whereas a decision requires only a distribution over a few options, so the output is one score per option rather than a set of predicted triples, and there is no dynamics step. And whereas a world model's state may hold several entities that share attributes, each scope in a decision question describes a single entity, which allows a triple to be represented as one token rather than three. We keep the representation and the principle of order-free sets, and change the output and the tokenization to fit the task.

4.2 Representation

Each question is rewritten as an unordered set of triples of the form scope, attribute and value, such as `(town, food, low)` or `(district, rent, high)`. A single function derives these triples from the schema that already defines each kind of question. Categorical fields become one triple each; integer fields with a small range are treated as categories; other numeric fields carry their value, compressed as sign(x)·ln(1+|x|), alongside the triple; and lists such as the active rules contribute one triple per item, together with a level where the item has one. The names of towns, districts and proposals are discarded, since nothing learned from a name transfers to another town. A value absent from the vocabulary maps to a shared unknown token, which also signals that the student is outside its experience. Because the representation follows from the schema, a new kind of question requires no new feature code.

4.3 The set model

The student is a set model with a single decision layer (Figure 2). Each triple is embedded as one vector, the sum of embeddings for its scope, attribute and value, with numeric values added along a learned direction for their attribute. The question's own triples, together with one token for each permitted answer, are the inputs of the decision layer. Each answer token is built from an embedding specific to that kind's option and one shared across kinds, so that an option such as leaving town draws on what is known about leaving in other decisions. In the decision layer, the question and answer tokens attend to one another and to the facts of the snapshot, which enter as their embeddings alone, normalised and projected into the keys and values that attention reads. Each answer token ends in one score, and a softmax over the permitted options gives the distribution.

Two properties of this layout make the model cheap to run. The snapshot's keys and values depend on nothing but the snapshot, so the browser computes them once and reuses them for every question that shares it, whatever its kind. And only the answer tokens reach the scores, so the layer computes their rows and no others.

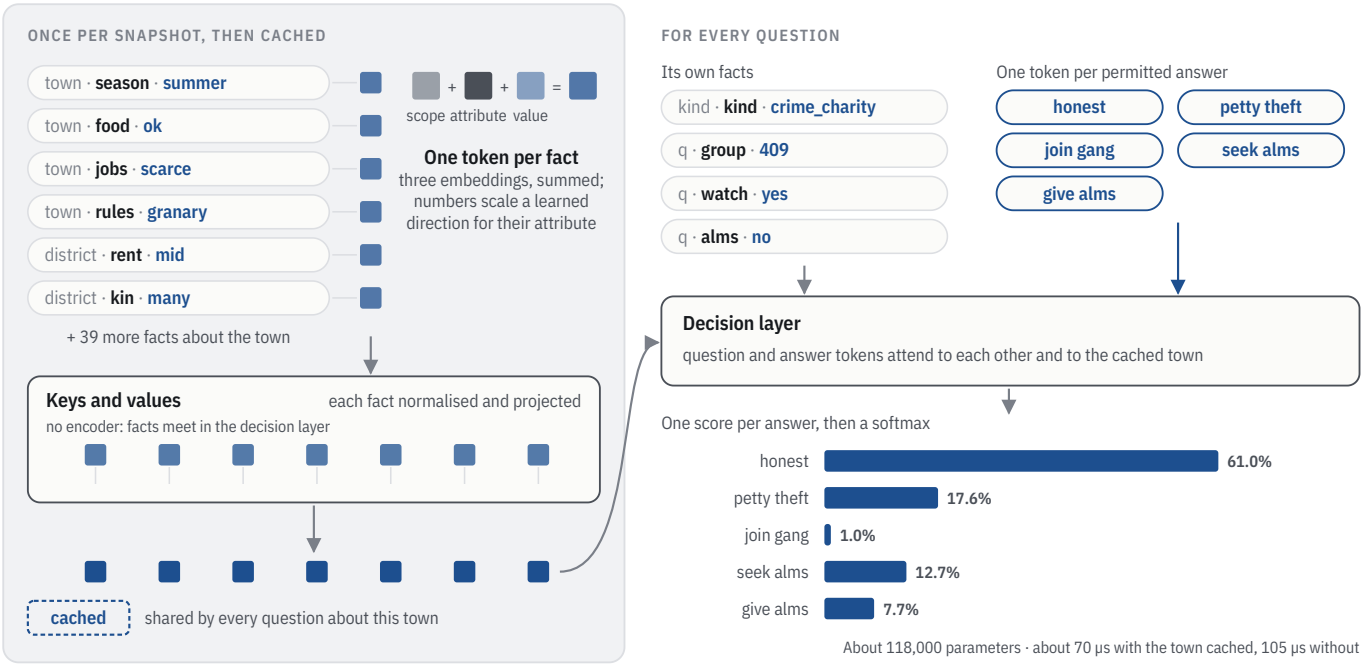


Figure 2. The set model, on one question: whether a group of 409 residents stays honest, turns to petty theft, joins a gang, seeks alms or gives them. Each fact becomes one token. The town's facts are projected once into keys and values and cached; each question runs only the decision layer, in which one token per permitted answer attends to the question's facts and to the cached town. The bars are the deployed student's answer to this question.

The snapshot has no encoder of its own. A variant that first relates the snapshot's facts to one another with an attention layer, as the Triple World Model does, is less accurate and three times slower when a town is first seen (Section 6.2): the decision layer, which already reads every fact from each answer token, relates them as each answer requires. Summing a triple's three embeddings into one token also departs from the Triple World Model, which gives each of entity, attribute and value its own token and binds them with position embeddings. In Hearthvale each scope describes one entity, so the sum already binds a value to its attribute and scope, and the single-token form needs a third of the tokens. We nevertheless retain the original layout as one of the alternatives, alongside three models representing the traditions described in Section 2: a multilayer perceptron over the summed triple embeddings, with dropout tuned for a fair comparison; a factorization

machine whose option scores weight pairwise, and optionally third-order, interactions between triples, computed in linear time; and gradient-boosted trees, fitted separately for each kind on one-hot features.

5 Method

5.1 Data

Training data is gathered in two stages. The first provides breadth. Sixty-four towns are grown automatically to their 150th year across seven kinds of terrain, each governed by a stand-in council that rules on every proposal in one of ten political styles, from frugal to mercantile to pious. A recording layer captures every question exactly as the game client would pose it and answers it with the fallback rules so that the towns can proceed. Because these simulations do not step each resident individually, residents' immediate needs are static, and copies of each resident question with randomly drawn need levels let the student see the full range. These runs produce several million distinct questions spanning hamlets to late-era cities.

The second stage follows DAgger. Sixteen further towns are grown under the control of a student, so that the data also covers the states a student's own decisions lead to, which differ from those reached under any other policy. In both stages the teacher answers a sample of the questions through the game's own request path, so that the prompts it sees are identical to those the game sends in play. Sampling takes up to 15,000 questions of each kind, more for the frequent routine decision, and draws several questions from each snapshot, which lets the teacher's server reuse its encoding of a shared snapshot. A third set, used only for evaluation, comes from four towns that the teacher itself governs for 150 years. In all, the teacher labels 460,701 questions, and the deployed student is also trained on the questions from play that the learning loop gathers (Section 5.5).

5.2 Training

Every neural student minimises the divergence from the teacher's distribution over the permitted options. Each question is weighted by the square root of how often the simulated towns asked it, so that common situations count for more without overwhelming rare ones. Training uses the AdamW optimiser with a one-cycle learning-rate schedule for twenty epochs, and keeps the epoch with the lowest error on held-out towns. The cheaper students can additionally learn from a trained set model: it labels 1.2 million further questions from the training towns, and the factorization machine and perceptron are trained on these labels together with the teacher's.

5.3 Evaluation protocol

Average error on held-out data is a necessary test but an insufficient one, so we evaluate the students in four increasingly demanding ways. The first measures error on towns excluded from training. Twenty per cent of towns are held out entirely, and because snapshots change every season, nearly every held-out question presents a snapshot never seen in training.

The second targets compositional generalization directly. For every pair of a question fact and a snapshot fact, we compare the teacher's average answer when both are present with an additive prediction from each fact alone, and choose four pairs whose joint effect departs most from additivity, together with two weakly interacting pairs as controls. Every training example containing both facts of a pair is removed, while examples containing either fact alone are kept, and we measure error on exactly the removed examples.

The third measures the student on questions from towns the teacher governs. These towns are grown headless, with residents' needs synthesised as in the first stage, so they are not play by people; but their states are those a town reaches under the teacher's policy, which is what a faithful student must handle, and they resemble the deployed game far more closely than towns run by the fallback rules. We call them teacher-governed towns.

Alongside the divergence, weighted both uniformly and by how often each question is actually asked, we compare the average probability the student and the teacher assign to each answer, and the ratio of the two for answers the teacher gives rarely.

The fourth compares complete towns over 150 years. Because the game samples its answers with a seeded random generator, the same town governed twice by the same model replays identically; to measure the natural variation of a town, we re-seed its random stream immediately after creation, so that runs share a map and a starting point and differ only in their draws. Who plays the council matters as well. A stand-in council that rules on every card the moment it appears makes every run play alike, so these towns are played instead by a simulated player: it opens the docket in some months but not others, takes up only part of what it sees, lets the rest lapse at their deadline as an absent player would, and now and then chooses at random. Its engagement is drawn afresh for each run, and it keeps its own random stream, so that who answers the residents does not change how the player behaves. Four towns are grown six times each under the teacher, eight times each under the student and three times under the fallback rules; the same comparison with the stand-in council is reported alongside. For each outcome we report two quantities. The first is location: the mean difference between student and teacher runs of the same town, averaged over towns, with a bootstrap interval over runs and a permutation test that exchanges the labels of runs within each town. The second is spread: the pooled within-town standard deviation of the student's runs divided by that of the teacher's, which is about one when the student's towns vary as much as the teacher's. Both are needed. A single ratio of typical distances, student to teacher over teacher to teacher, rewards a student with no variation at all: one that always reproduced the teacher's median town would score about 0.71 on a normally distributed outcome and appear closer to the teacher than the teacher is to itself.

5.4 Running in the browser

The student ships as a compact bundle in which each weight matrix is stored as eight-bit integers with one scale per row, a change that alters answers by less than one ten-thousandth of a nat. The bundle carries a set of test questions with the answers its training framework gives, and it must reproduce them in JavaScript to within seven parts in ten million before it is published. The runtime lives in the simulation's worker thread and keeps two caches. The first holds each snapshot's keys and values, shared across every kind of question that uses the snapshot. The second holds answers: a town asks most questions two or three times, the daily routine more than three, and an answer depends only on its question, so each distinct question is computed once and later askings are served from a bounded cache; the game samples from the distribution afterwards, so the cache changes nothing it does. Because a month's opening sweep can pose dozens of questions in a single simulation step, the runtime limits the student to a few milliseconds of work at a time and completes the remainder on later steps, much as answers from the network would arrive. Every town, new or saved, opens on the latest published student and keeps it for the session.

5.5 Learning after release

A fixed student would eventually meet towns its training never covered, so the system keeps learning from the teacher. A small auxiliary network, trained on held-out towns, predicts the student's own error on each question from the answer tokens' final states, the answer's entropy and the share of unknown inputs; a single scale fitted on the held-out data makes its output read in nats. Questions whose predicted error lies in the top twentieth, questions containing values the student has never seen, and questions of kinds it does not answer are sent, deduplicated and without any player identifier, to the game's server. The server validates them, records a salted hash of each client so that distinct reports can be counted without identifying anyone, and obtains the teacher's answer while the GPU is idle.

A job that runs once a day turns these reports into the next version when at least fifty new labelled questions are waiting. A single report is enough for a question to be used. Labels come only from the teacher, so one player can at most direct the teacher's attention to unusual states, and the comparisons described below judge the result; requiring several players to report the same question would, in a game whose states rarely repeat exactly, discard nearly everything. What one player can contribute is bounded instead. A cycle takes at most 400 questions from any one client, in the order they arrived, and defers the rest to later cycles; questions whose categorical facts form more than two pairs never produced by any simulated town may make up at most a quarter of a cycle unless three clients reported them; and questions already trained on are skipped. A cycle can also take rows gathered on purpose, such as teacher-labelled questions from young towns the student governs.

The next version starts from the current one, appending new vocabulary entries initialised from the unknown token rather than retraining from scratch; when the architecture itself changes, it is instead trained from scratch on all the data. Its error estimate is retrained, and it is compared with the current version on error in teacher-governed towns, overall and by traffic, error on every frequently asked kind, the largest bias on any answer overall and in hamlets and villages, rare-answer ratios, error on held-out new questions, browser parity, size, speed and a set of short simulated towns. The candidate is published only if it passes every comparison.

When the teacher is available, the browser also refers the questions it is least certain of directly to it. Each player may send at most one such question per second, always the least certain first, so the share referred falls as the game speeds up; the server enforces a further budget shared by all players and refuses the excess with a time to wait, in which case the student's own answer, already computed, is used.

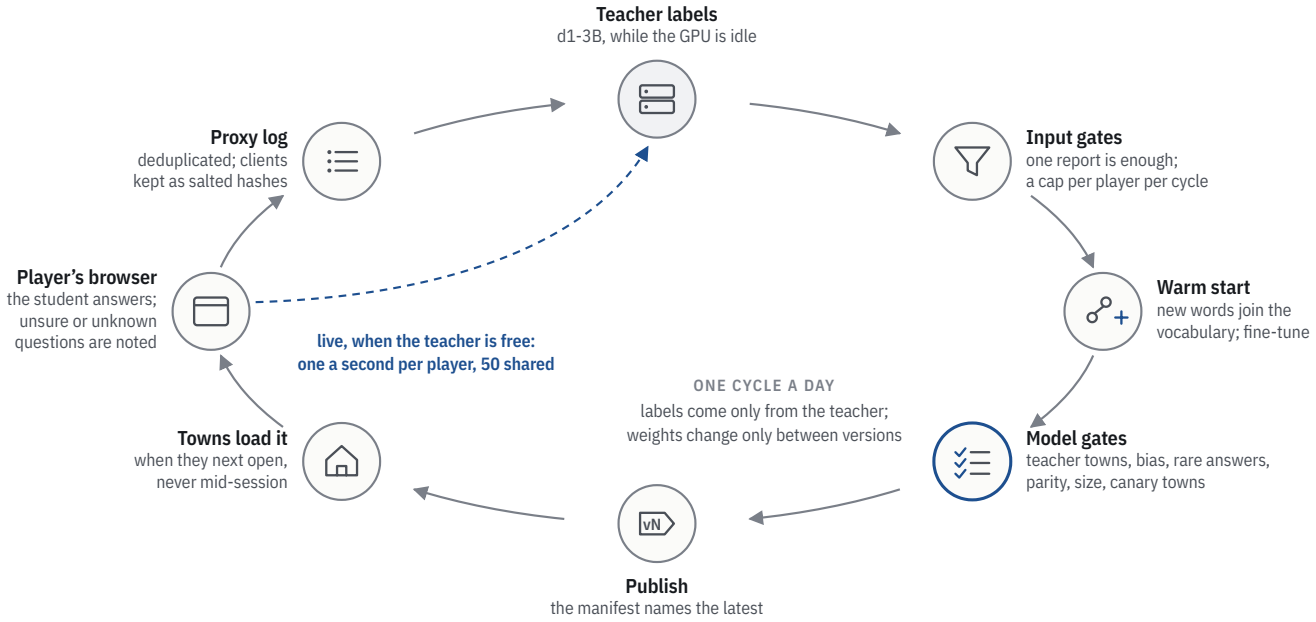


Figure 3. The continual-learning loop. Labels come only from the teacher, and one player's report is enough, within a per-player cap; weights change only between versions, and a town takes the latest version when it next opens.

6 Results

6.1 Architectures compared

The five families are compared on the broad first-stage data under identical conditions (Table 2, Figure 4). The set model reproduces the teacher most closely, with a divergence of 0.025 nats on towns it never saw and agreement on the most likely answer in 86% of questions. Gradient-boosted trees come close in accuracy, but a model covering every kind requires about 12.7 megabytes of trees, some forty times the size of the set model. The factorization machine is somewhat less accurate yet exceptionally cheap, and learning from the set model's labels closes part of the gap. The multilayer perceptron, even after tuning, is the least accurate by a wide margin. Within the set model, a single layer in each stage costs almost nothing in accuracy against two and takes about a third of the time.

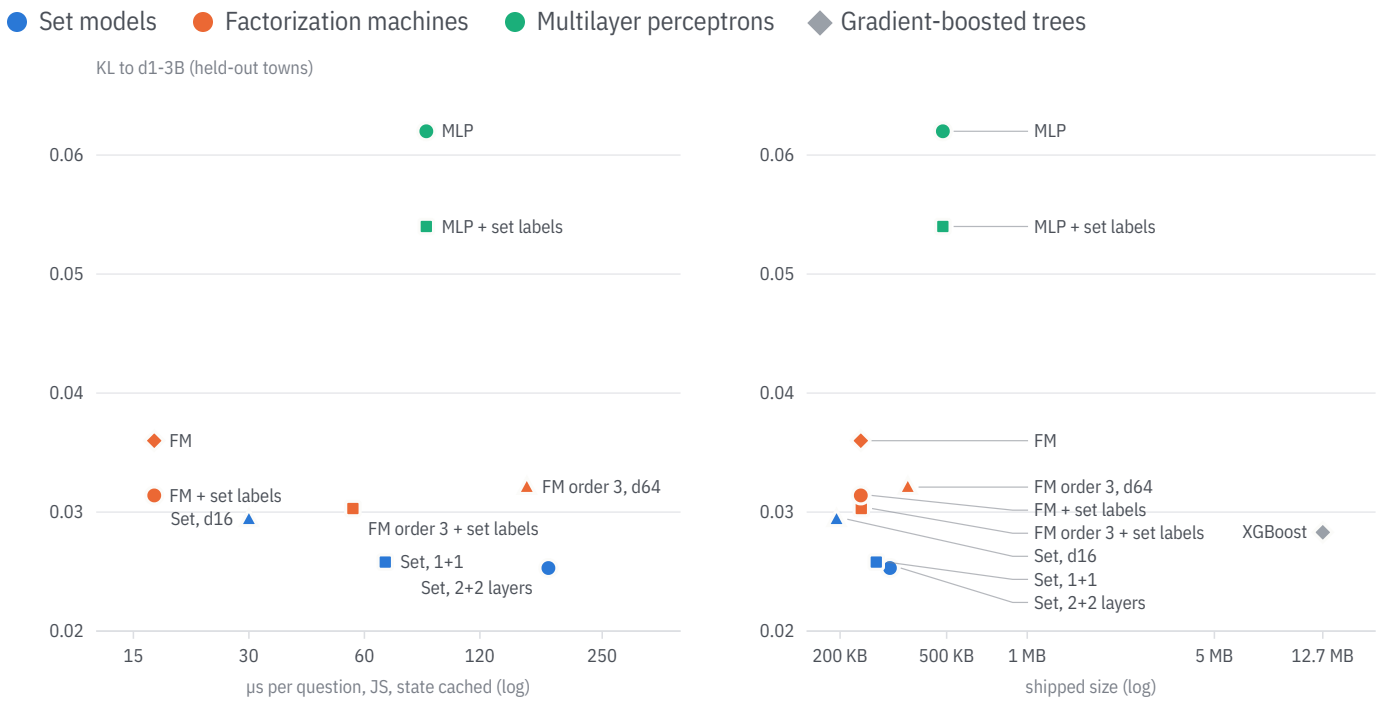


Figure 4. Divergence from the teacher on held-out towns of the first stage against cost per question in the browser with the town cached (left) and the size of the bundle (right). Lower and further left is better. Each model keeps its colour and marker in both panels. Gradient-boosted trees have no browser implementation and appear only on the right.

Table 2. Architectures compared on 30,667 questions from held-out towns of the first stage. The baseline, which answers every question with its kind's average distribution, scores 0.150.

Student	Divergence (nats)	Top answer agrees	Bundle size	Time per question, town cached
Set model, two layers per stage	0.0253	85.7%	307 KB	~180 µs
Set model, one layer per stage	**0.0258**	**85.3%**	**273 KB**	**~70 µs**
Set model, small (d = 16)	0.0294	84.4%	194 KB	~30 µs
Gradient-boosted trees	0.0283	85.4%	~12.7 MB	not built
Factorization machine, with set-model labels	0.0314	84.0%	239 KB	~17 µs
Factorization machine	0.0360	82.6%	239 KB	~17 µs
Multilayer perceptron, best of four	0.0620	80.5%	484 KB	~87 µs

6.2 Which parts of the set model earn their cost

Most of the set model's parameters are its vocabulary, and every variant in Table 4 keeps it whole: the question is whether the layers around it can be narrower or shallower without loss. The variants are trained with the same recipe on both stages of data, changing one thing at a time from a set model with a snapshot encoder, and are timed on the same 2,000 questions from teacher-governed towns.

Removing the snapshot encoder improves every measure. The model without it has 5% lower error on held-out towns, 4% lower on teacher-governed towns and 9% lower when weighted by traffic, with a smaller largest bias. It answers a cached question as quickly and an uncached one in a third of the time, and has 7% fewer parameters. A plausible reading is that the encoder mixes a town's facts before any question is asked, spending time on a step the decision layer does better for itself. This is the deployed configuration.

Narrowing the model trades accuracy for speed roughly in proportion. At a width of 24 the error rises by about 5% on held-out towns and 8% on teacher-governed towns, and cached answers come a third faster; at 20 the error rises by 8% to 12% for answers nearly twice as fast. A single attention head saves almost nothing and costs a little accuracy. Training the narrower model on the set model's labels as well recovers most of its loss, but those labels bring a larger vocabulary with them, and the bundle grows by half.

Table 4. Variants of the set model, full vocabulary, the same recipe and data. Held-out towns: the training split; teacher-governed towns: 40,119 questions, overall and weighted by traffic, with the largest bias on any answer of a frequent kind in points. Times per question on 2,000 teacher-governed questions on one core, with the town cached and without; download is the gzipped bundle. ff: feed-forward width.

Student	Parameters	Held-out towns	Teacher-governed	By traffic	Largest bias	Cached	Uncached	Download
With a snapshot encoder (width 32, ff 64)	125,089	0.0217	0.0204	0.0122	2.1	67 µs	314 µs	172 KB
No snapshot encoder (deployed)	**116,545**	**0.0206**	**0.0196**	**0.0111**	**2.0**	**67 µs**	**105 µs**	**162 KB**
No encoder, ff 32	114,465	0.0207	0.0199	0.0125	3.8	60 µs	97 µs	160 KB
ff 32	120,929	0.0219	0.0211	0.0115	2.0	60 µs	269 µs	168 KB
Width 24, ff 48	89,977	0.0229	0.0220	0.0128	1.7	45 µs	203 µs	135 KB
Width 24, ff 48, one head	89,977	0.0233	0.0230	0.0132	1.8	45 µs	194 µs	135 KB
Width 20, ff 40	73,381	0.0235	0.0229	0.0128	2.4	36 µs	155 µs	118 KB
Width 24, ff 48, set-model labels	136,081	0.0210	0.0209	0.0122	1.5	46 µs	201 µs	200 KB

6.3 Compositional generalization

The test of withheld pairs separates the architectures sharply and bears out the reasoning of Section 4.1. On combinations of facts it never saw together, the set model's error is only 4% higher than on ordinary questions. The factorization machines lose between 13% and 15%, consistent with their design, and they perform best of all on the purest two-way interactions, such as craftsmen weighing a proposal that favours craft. The multilayer perceptron's error, by contrast, nearly quadruples, and on one pair, young people choosing a trade when farms offer no work and wellbeing is poor, its answers are scarcely better than the kind's average.

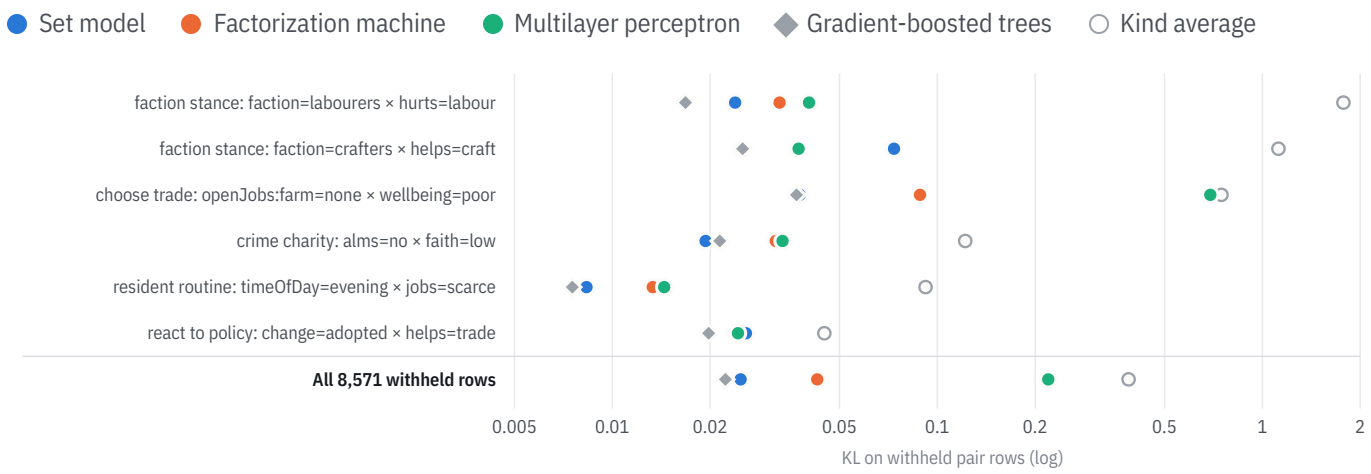


Figure 5. Divergence on questions whose pair of facts was withheld from training (logarithmic scale). Hollow markers show the kind-average baseline.

6.4 The deployed student

Table 3 reports the deployed student, trained on both stages of data and on questions from play, against the main alternatives on held-out towns and on the teacher-governed towns, which play no part in training. It reaches 0.020 nats on teacher-governed towns, and 0.011 when each question is weighted by how often the game actually asks it, since the most frequent decisions, such as a resident's daily routine, are also among the easiest. Because the game samples from these distributions rather than taking the most likely answer, we also compare the average probability the student and the teacher assign to each answer. For the daily routine, which accounts for nearly three quarters of all questions, every answer lies within 0.6 percentage points of the teacher (Figure 6); across all frequently asked kinds no answer differs by more than 2.2 points, and the rate of every rare outcome lies within two thirds and one and a half times the teacher's. Training without the second, on-policy stage leaves the error on teacher-governed towns about a fifth higher, concentrated in household decisions (Table A2), which supports gathering states from the student's own play.

Table 3. The deployed student and the main alternatives, trained on both stages of data. Teacher-governed towns: 40,119 questions from four headless towns the teacher governed, never used in training.

Student	Held-out towns	Top answer agrees	Teacher-governed towns	Same, by traffic
Set model, no snapshot encoder (deployed)	**0.0206**	**86.9%**	**0.0203**	**0.0114**
Set model with a snapshot encoder	0.0217	86.4%	0.0204	0.0122
Set model, small (width 16)	0.0256	84.7%	0.0265	0.0155
Factorization machine, with set-model labels	0.0256	85.4%	0.0265	0.0156

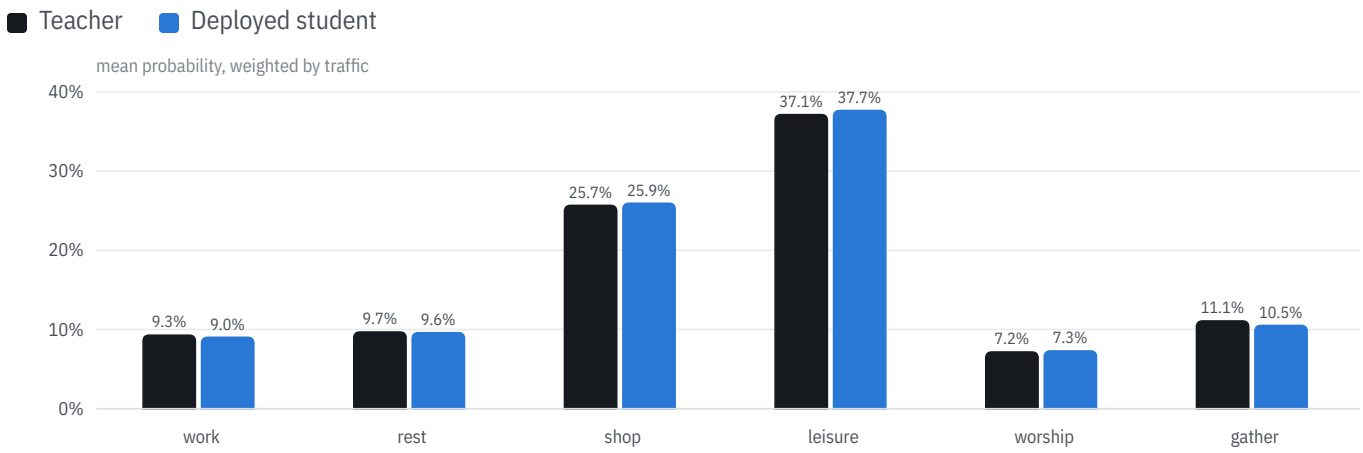


Figure 6. Average probability of each answer to a resident's daily routine on questions from teacher-governed towns, weighted by how often each question is asked: the teacher and the deployed student.

6.5 Behaviour of complete towns

The decisive test is whether towns behave alike. Chance alone shapes a town's history strongly: grown six times under the teacher, one town ends its 150th year with anywhere from 252 to 6,594 inhabitants. Table A3 compares runs of the same towns, played by the simulated player, under the deployed student and under the teacher.

From the middle of the game on, the student's towns cannot be told apart from the teacher's. Population at the 100th and 150th years, the share of residents at work in the morning and the evening, and wellbeing give permutation p-values between 0.22 and 0.82. Early on they can: at the 50th year the student's towns are about a third smaller (−0.43 in log population,

90% interval −0.65 to −0.22, p = 0.004), a lag they make up by the 100th year. Section 6.6 traces it to a single decision. The fallback rules differ on everything: by the 150th year their towns are about six and a half times the teacher's, with 34 more points of the population at work in the morning.

In spread, the student's towns vary about as much as the teacher's. The ratio of within-town standard deviations is between 1.1 and 1.2 for population at the 50th and 150th years, the evening work share and wellbeing, with intervals that include one. Two outcomes vary less: population at the 100th year (0.71, interval 0.55 to 1.02) and the morning work share (0.60, interval 0.46 to 0.86). With the stand-in council the student's towns appear narrower on almost every outcome (Table A3, lower part), but most of that narrowing belongs to the council, which plays every run alike, rather than to the student. What remains in the morning work share does not lie in individual answers: across the teacher-governed questions the student's probability of work in the morning varies 0.95 times as much as the teacher's. It may follow from the early lag, since towns that grow more slowly at first have less time to diverge.

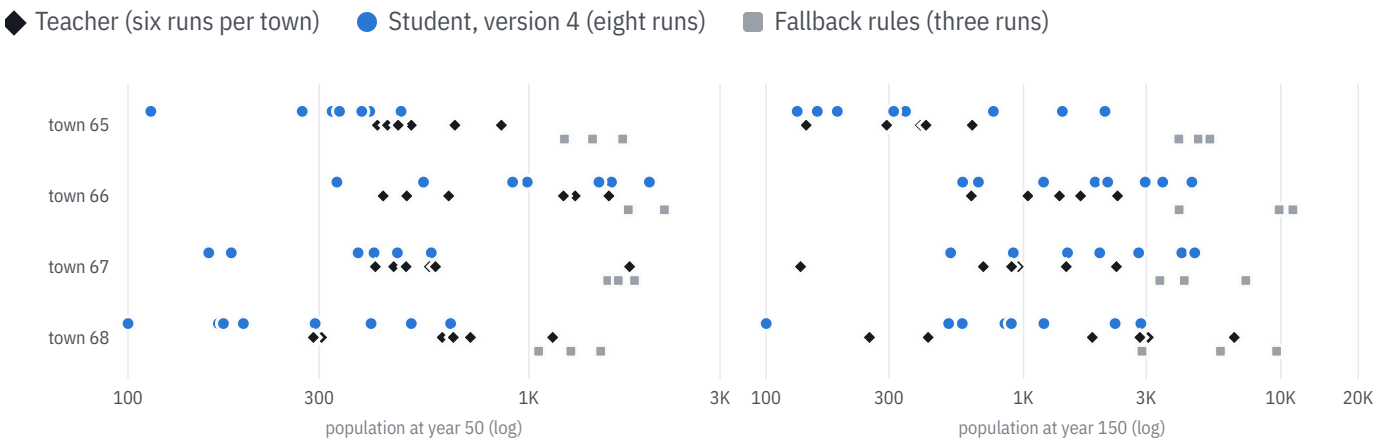


Figure 7. Population in the 50th year (left) and the 150th (right) for every run of four towns unseen in training, played by the simulated player (logarithmic scale). Runs of a town share its map and starting point and differ only in their random draws. The student's towns trail the teacher's at the 50th year and have caught up by the 150th.

6.6 Tracing the early lag

The game does not put the choice of where newcomers settle to the model, so the lag of Section 6.5 must come from another decision. Comparing each answer's average probability under the student and the teacher on questions from the hamlets and villages of the teacher-governed towns, one decision stands out: a household's choice whether to stay. There the student gives staying 64.5% against the teacher's 69.5%, and emigrating 10.9% against 9.3%; no other decision bearing on growth, among arrivals, homes, land, enterprises and trades, differs by more than about two points. In older towns the same decision shows no lean (64.3% against 63.8%). Young towns are a small part of what the student learns from, about one question in six of the teacher-governed set, because a town does not stay small for long, and a few more households leaving each year compound over decades. The remedy follows the on-policy principle of Section 5.1: questions from young towns the student governs, labelled by the teacher and added to the learning loop, together with the loop's comparison of the largest answer bias in hamlets and villages, so that the lean cannot return unnoticed.

Handing this one decision back to the teacher confirms the diagnosis. In sixteen runs of four towns played for fifty years, with the student answering every other question, the towns reach 81% of the population of the teacher-governed runs at the 50th year (geometric mean over matched runs), against 65% when the student answers everything; at the 30th year they are level with the teacher's, at 278 inhabitants on average against 260, where the student alone reaches 213. The fifth version of the student was trained with 5,840 questions of this kind from young towns, labelled by the teacher (Section 6.8).

The same comparison shows where answers are too even. In the evening the student's probability of work moves only 0.37 times as far as the teacher's from one set of circumstances to another. At 3 to 4 percent of evening answers the compression is too small to change towns, but it is the

kind of quiet distortion that a standing comparison of answers by kind, town stage and responsiveness should watch as both the game and its students change.

6.7 Cost

The deployed student answers a question in about 70 microseconds on a single processor core when its town is cached and about 105 when it is not, against five to nine milliseconds of GPU time per question for the teacher. Its bundle is 353 kilobytes, 165 when compressed for download: 175 of model weights, 12 for the error estimate, 144 of vocabulary and 20 of test questions. Grown end to end, a town governed by the student reaches its 150th year in a few minutes, against ten to thirty for a town waiting on the teacher, and costs little more than running with no model at all, because the simulation itself dominates. The heaviest single steps carry several dozen questions at the opening of a month, which the per-step budget spreads over subsequent steps.

6.8 Confidence and the learning loop

The student's estimate of its own error is useful. Its rank correlation with the actual error on teacher-governed towns is 0.53; marking the 5% of questions it judges most uncertain captures 46% of its worst errors, nine times what random selection would achieve (Table A4). The questions players' browsers report on this basis carry about four times the average error. The loop has run four cycles on questions from play: one on a single browser session and three on questions from one player on the game's server. The third retrained the student from scratch in the configuration of Section 6.2. Each of the first three passed every comparison and lowered the error on the new questions it held out, by between a tenth and nearly a third, without raising the error in teacher-governed towns by more than 2%. The fourth added 400 new questions from play and the 5,840 young-town questions of Section 6.6. It too passed every comparison: the error on the new questions it held out fell by an eighth (0.088 to 0.077 nats), the error

on questions from play from 0.0203 to 0.0201, and the largest bias on any answer of a frequent kind from 2.2 to 1.9 points. It is the version the game now serves; whether it closes the early lag of complete towns is being measured.

7 Discussion

7.1 What the triple route contributes

The results bear out the properties that motivate the approach. The ordering of the architectures follows from the structure of the decisions: most of the teacher's answers depend on individual facts and on pairs of facts, such as a faction and the interests a proposal favours, or a trade and the jobs available. Attention and factorization both compute products of features, whereas a multilayer perceptron over a sum of embeddings must approximate such products with successive nonlinear layers, which demands more data than the problem supplies. Factorization machines capture pairs efficiently but assign each pair a fixed strength, while attention can make an interaction depend on a third fact and disregard irrelevant ones. The resulting order, set model, then factorization machine, then perceptron, mirrors how much multiplicative structure each design builds in, and the withheld-pair test shows the same order with the largest margins.

The economy of the representation is equally valuable in practice. A single set model of about 118,000 parameters answers every kind of question; the snapshot's keys and values, shared across questions, keep the common case cheap; and because new facts are new tokens, the learning loop extends the vocabulary and fine-tunes rather than rebuilding. These are the properties the Triple World Model was designed to demonstrate, and they transfer to a commercial setting with little modification.

7.2 Why the student is fast

The student's speed does not come from a clever kernel. It follows from four decisions, each of which removes work that a general model must do, and together they explain how a model of about 118,000 parameters answers in well under a tenth of a millisecond on one processor core.

The first is that the input is never text. The teacher reads some three hundred tokens of prose for each question and must interpret them with three billion parameters before it can weigh the options. The student receives the same information as about fifty facts, each already a single token, so reading a question reduces to looking up three vectors per fact and adding them. This is also why most of the model's parameters cost nothing to run: the table of value embeddings holds nearly three quarters of them, and a question touches only the rows its facts name (Table 5).

The second is that the town is read once. A town's facts change once a season, while dozens of questions of every kind are asked about it each month. The keys and values the decision layer reads from them are computed when a snapshot is first seen and reused by every question until the season turns, so a question pays only for its own few facts and its answers.

The third is that the answers are tokens rather than output units, and only they are computed to the end. Each permitted answer enters the decision layer as a token built from an embedding of its own and one shared across kinds, and leaves it as a single score. A question with five options costs five tokens' worth of work in the final rows, one model serves all twenty-two kinds without a separate output for each, and because attention multiplies features, the interactions between pairs of facts that drive most answers come from a single narrow layer rather than from depth.

The fourth is that the model is small enough to need no special machinery. With a width of 32 and one layer, the arithmetic per question suits plain JavaScript loops over typed arrays. The runtime needs no graphics

processor, no WebAssembly and no framework, so it never competes with the game's renderer; it runs in the simulation's own worker thread, answers each distinct question once, and spreads a burst of questions over successive simulation steps rather than stalling any one of them. Around the forward pass the same principle holds: each snapshot is validated and keyed once rather than for every question, and the option tokens' first-layer work is memoized, which together more than halve the student's total time when it replays the 18,040 questions of a forty-year town, to roughly 70 microseconds a question with every overhead included.

The same structure keeps training cheap. Learning the teacher's full distribution gives each example far more information than a single label would; the on-policy stage spends labels where the student's own play leads; and a new version starts from the old one, adding vocabulary rather than retraining. A complete cycle of the learning loop, from labels to the final comparison, takes under twenty minutes on four processor cores.

Set against the alternatives, these decisions account for the results of Section 6.1. The factorization machine is faster still, at about 17 microseconds, because it computes only fixed pairwise products, but it cannot let a third fact change the strength of a pair, and it pays for that in accuracy. Gradient-boosted trees come close in accuracy but share nothing across kinds and need forty times the space. The perceptron must learn through depth the products that attention computes directly. The set model sits where the structure of the problem places it: the most accurate student, at a cost that is a small fraction of the game's own simulation.

Table 5. Where the deployed student's parameters go, and which of them each question uses. Times are per question on one processor core of the test machine.

Part	Parameters	Share	Work per question
Value embeddings	88,928	73.8%	one row looked up per fact
Scope, attribute, number and option embeddings	16,320	13.5%	rows looked up
Decision layer and scores	**13,025**	**10.8%**	**about 70 µs with the town's keys and values cached; about 105 µs when they are computed**
Error estimate	2,209	1.8%	a small network on the answer tokens' final states

7.3 The three-token layout

The Triple World Model's original layout, which writes each triple as three separate tokens, learns more slowly here and is set aside. In its original setting a state holds several entities sharing the same attributes, and relating values to entities across tokens is the problem attention solves. In Hearthvale each scope describes a single entity, so summing a triple's three embeddings already binds them, and the three-token form must instead learn that binding from position, at three times the cost per question. The representation transfers; the tokenization does not need to.

7.4 Evaluating a policy that acts repeatedly

The evaluation design deserves emphasis. Because a game samples from the student's distribution at every step for a century, a small and consistent bias on one answer can alter a town more than larger random errors, and average divergence alone cannot reveal it. For this reason the evaluation compares the probability of each answer as well as the overall divergence, measures the student on the states its own decisions produce, and compares complete towns with the teacher's in both average and spread. The last point is easy to get wrong: a single ratio of distances between runs cannot distinguish a faithful student from one whose towns are merely less varied. How the player is simulated matters as much. A council that rules on every card at once makes every run play alike and narrows the towns' spread by itself, and outcomes must be read by stage, since a lag at the 50th year is invisible at the 150th. We regard these measures as necessary for distilling any policy that acts repeatedly within a simulation.

7.5 The teacher's character

The experiments reveal something about the teacher itself. Its answers keep towns markedly smaller than the fallback rules do, chiefly because it sends few residents to work. The student copies this faithfully, which is the

intended behaviour, but it is a property of the teacher that the game's designers may wish to examine, and one that a distilled student makes cheap to study.

7.6 Limitations

The comparison of complete towns rests on four towns and six teacher runs of each, enough to find a lag at the 50th year but not to estimate the teacher's spread precisely; it uses a version of the simulation without the foundation costs of the deployed game, and is being repeated on eight towns. The simulated player is itself a model of how people play. Simulated towns do not step every individual resident, so residents' needs are synthesised, and the teacher-governed towns are headless; the questions from play come from a few players so far. Timings come from a single fast machine, and a mid-range phone will be several times slower, although the per-step budget accommodates this. The withheld-pair test covers six pairs chosen for strong interactions, and the error estimate misses more than half of the worst questions when it marks only a twentieth of them.

8 Conclusion

A small model of structured triples can take the place of a large language-model decision policy for the great majority of a game's decisions. It matches the teacher closely, generalizes to unfamiliar combinations of circumstances, reproduces the course of the teacher's towns from the middle of the game on, and costs a small fraction of the GPU time it replaces. Three elements make this possible: a representation, inherited from the Triple World Model, that exposes the structure of each question; an evaluation grounded in the states the deployed policy reaches and in complete towns rather than in average error alone; and a loop through which the student keeps learning from its teacher after release. The same approach should extend to other games and simulations that rely on

language models to animate their agents, wherever the questions put to those models are structured and the cost of answering them scales with the audience.

References

1. G. Hinton, O. Vinyals and J. Dean. Distilling the knowledge in a neural network. *NIPS Deep Learning Workshop*, 2015.
2. A. A. Rusu et al. Policy distillation. *International Conference on Learning Representations*, 2016.
3. T. Chen and C. Guestrin. XGBoost: A scalable tree boosting system. *KDD*, 2016.
4. Y. Gorishniy, I. Rubachev, V. Khrulkov and A. Babenko. Revisiting deep learning models for tabular data. *NeurIPS*, 2021.
5. A. Beutel et al. Latent cross: Making use of context in recurrent recommender systems. *WSDM*, 2018.
6. S. Rendle. Factorization machines. *IEEE International Conference on Data Mining*, 2010.
7. M. Zaheer et al. Deep sets. *NeurIPS*, 2017; J. Lee et al. Set Transformer. *ICML*, 2019.
8. S. Ross, G. Gordon and D. Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. *AISTATS*, 2011.
9. Y. Geifman and R. El-Yaniv. Selective classification for deep neural networks. *NeurIPS*, 2017; D. Madras, T. Pitassi and R. Zemel. Predict responsibly: Improving fairness and accuracy by learning to defer. *NeurIPS*, 2018.
10. Cloudflare. Clef-flash model card. huggingface.co/Cloudflare/clef-flash, 2026.
11. J. Palmer. Kev: open decision models. github.com/jaredpalmer/kev, 2026.
12. Convai Innovations. Laya: a non-autoregressive System 1 decision engine. github.com/NandhaKishorM/laya, 2026.

Appendix A. Detailed results

Table A1. Divergence on held-out towns and on withheld pairs, from a training run with the six pairs removed.

Student	Held-out towns	Withheld pairs	Ratio
Set model	**0.0238**	**0.0248**	**1.04×**
Set model, small	0.0288	0.0332	1.15×
Factorization machine, third order	0.0324	0.0366	1.13×
Factorization machine	0.0372	0.0427	1.15×
Multilayer perceptron	0.0583	0.2194	3.76×
Gradient-boosted trees (five kinds)	—	0.0223	—

Table A2. Divergence on questions from teacher-governed towns, for students trained on the first stage of data only and on both stages, overall and weighted by how often each question is asked.

Student	Held-out towns	First stage: teacher-governed	First stage: by traffic	Both stages: teacher-governed	Both stages: by traffic
Set model, two layers per stage	0.0253	0.0232	0.0145	—	—
Set model, one layer per stage (shipped)	**0.0258**	**0.0242**	**0.0164**	**0.0204**	**0.0122**
Set model, small	0.0294	0.0292	0.0184	0.0265	0.0155
Factorization machine	0.0314	0.0321	0.0185	0.0265	0.0156
Factorization machine, third order	0.0303	0.0311	0.0193	—	—
Multilayer perceptron	0.0620	0.0613	0.0336	—	—

Table A3. Complete towns: the student's and the fallback rules' runs against the teacher's runs of the same four towns. Shift is the mean difference from the teacher, with a 90% bootstrap interval and a permutation p-value; spread is the within-town standard deviation over the teacher's, with a 90% interval. Upper part: the simulated player and the deployed student, six teacher runs per town. Lower part: the stand-in council and a student with a snapshot encoder, three teacher runs; its fallback rules ran once per town, so their spread is undefined. Population is compared on a logarithmic scale.

Outcome	Student shift	p	Student spread	Fallback shift	p	Fallback spread
With the simulated player						
Population, year 50	−0.43 [−0.65, −0.22]	0.004	1.14 [0.87, 1.65]	+0.91 [+0.75, +1.07]	<0.001	0.29 [0.15, 0.38]
Population, year 100	+0.05 [−0.27, +0.35]	0.818	0.71 [0.55, 1.02]	+1.42 [+1.10, +1.72]	<0.001	0.46 [0.22, 0.61]
Population, year 150	+0.16 [−0.20, +0.54]	0.527	1.11 [0.81, 1.73]	+1.87 [+1.54, +2.20]	<0.001	0.54 [0.25, 0.77]
Morning work share (points)	−0.79 [−1.89, +0.22]	0.222	0.60 [0.46, 0.86]	+34.13 [+33.03, +35.18]	<0.001	0.47 [0.23, 0.62]
Evening work share (points)	+0.04 [−0.08, +0.17]	0.623	1.12 [0.82, 1.58]	+5.57 [+5.40, +5.75]	<0.001	1.34 [0.67, 1.75]
Wellbeing, year 150	−0.83 [−2.72, +1.02]	0.528	1.18 [0.87, 1.90]	+0.83 [−4.38, +5.50]	0.772	2.94 [0.65, 4.62]
With the stand-in council						
Population, year 50	+0.22 [−0.07, +0.53]	0.251	0.75 [0.59, 1.60]	+1.01 [+0.76, +1.28]	0.024	—
Population, year 100	+0.07 [−0.39, +0.54]	0.775	0.41 [0.33, 0.93]	+1.44 [+0.97, +1.92]	0.030	—
Population, year 150	−0.46 [−0.89, −0.01]	0.075	0.63 [0.51, 1.34]	+1.40 [+0.98, +1.79]	0.034	—
Morning work share (points)	−0.85 [−1.96, +0.15]	0.163	0.65 [0.49, 2.19]	+36.93 [+35.93, +37.85]	0.004	—
Evening work share (points)	−0.20 [−0.53, +0.12]	0.201	0.25 [0.19, 0.80]	+5.61 [+5.26, +5.93]	0.004	—
Wellbeing, year 150	−2.05 [−7.10, +3.00]	0.608	1.10 [0.70, 4.15]	+7.42 [+3.75, +11.42]	0.195	—

Table A4. Questions marked by the error estimate, on teacher-governed towns.

Questions marked	Share of all error	Share of the worst 5%	Random marking
top 1%	9.8%	15.2%	1%
top 5%	**26.3%**	**46.4%**	**5%**
top 10%	39.0%	68.1%	10%

Table A5. Training and deployment settings.

Setting	Value
Set model (deployed)	width 32, no snapshot encoder, one decision layer, two attention heads, feed-forward width 64; 118,273 parameters and an error estimate of 2,209
Optimiser	AdamW, one-cycle schedule, peak learning rate 0.003, 20 epochs, batch 1,024; best epoch on held-out towns
Question weight	square root of how often simulated towns asked the question
Held-out towns	20% of towns, chosen by a hash of the town's seed
Bundle weights	8-bit integers with one scale per row; error estimate in 32-bit floats
Browser budget	4 ms of student work per turn of the event loop
Live referrals	at most one per second per player; a shared server budget, 50 per second by default
Loop intake	one report suffices; at most 400 questions per client per cycle, the rest deferred; unusual states at most a quarter of a cycle unless three clients report them; questions already trained on skipped; a cycle each day when 50 new labelled questions wait
Loop gates	no more than 3% worse on teacher-governed towns overall or by traffic; no frequent kind more than 20% worse; largest answer bias no more than 5 points, overall and in hamlets and villages; rare-answer ratios between 0.67 and 1.5

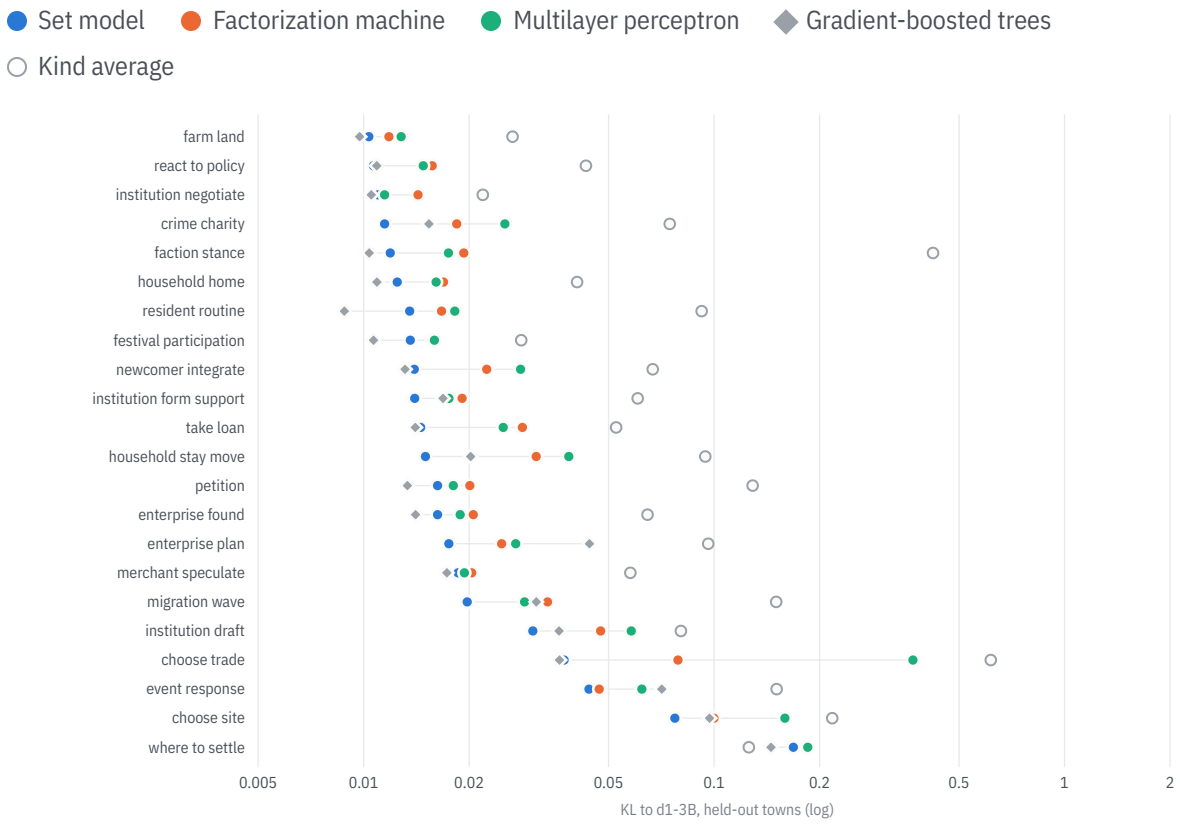


Figure A1. Divergence by kind of decision on held-out towns of the first stage (logarithmic scale). The hardest kinds are those whose options are free text that changes from question to question.

Code: Hearthvale `scripts/distill-*.js`, `src/decision-features.js`, `src/decision-student.js`, `src/decision-local.js`; the training code lives with the Triple World Model in `src/twm/decision/` and `scripts/train_decision_student.py`. Every published student matches its PyTorch original within 7×10^{-7} in the browser.